

AFRILANG Stdlib

std/c/graphprim

Bibliothèque AFRILANG `std/c/graphprim` (niveau complex, catégorie complex). Elle expose 7 fonction(s) :
primMst, primPa

```
`import "std/c/graphprim" . `use graphprim`
```

- `primMst(adj list number, n number) ? number`
- `primParents(adj list number, n number) ? list number`
- `primOrder(adj list number, n number) ? list number`
- `mstEdgeCount(adj list number, n number) ? number`
- `lightestEdge(adj list number, n number) ? number`
- `heaviestMstEdge(adj list number, n number) ? number`
- `isPrimConnected(adj list number, n number) ? bool`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/graphprim` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : primMst, primPa

```
`import "std/c/graphprim" . `use graphprim`  
- `primMst(adj list number, n number) ? number`  
- `primParents(adj list number, n number) ? list number`  
- `primOrder(adj list number, n number) ? list number`  
- `mstEdgeCount(adj list number, n number) ? number`  
- `lightestEdge(adj list number, n number) ? number`  
- `heaviestMstEdge(adj list number, n number) ? number`  
- `isPrimConnected(adj list number, n number) ? bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphprim"  
use graphprim
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? primMst(adj list number, n number) ? number  
? primParents(adj list number, n number) ? list  
? primOrder(adj list number, n number) ? list  
? mstEdgeCount(adj list number, n number) ? number  
? lightestEdge(adj list number, n number) ? number  
? heaviestMstEdge(adj list number, n number) ? number  
? isPrimConnected(adj list number, n number) ? bool
```

4. Exemples d'utilisation

```
import "std/c/graphprim"  
use graphprim  
  
create result = primMst(/* args */)   
say result  
  
create result = primParents(/* args */)   
say result  
  
create result = primOrder(/* args */)   
say result  
  
create result = mstEdgeCount(/* args */)   
say result  
  
create result = lightestEdge(/* args */)   
say result  
  
create result = heaviestMstEdge(/* args */)   
say result
```

5. Bonnes pratiques

- ? Préférez `import "std/c/graphprim"` en tête de fichier.
- ? Lancez `afrilang check` avant de livrer.
- ? Combinez avec les modules stdlib de la même catégorie.
- ? Voir /stdlib/ pour le source live et les démos playground.
- ? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

module graphprim

```
function primMst(adj list number, n number) returns number
end

function primParents(adj list number, n number) returns list number
end

function primOrder(adj list number, n number) returns list number
end

function mstEdgeCount(adj list number, n number) returns number
end

function lightestEdge(adj list number, n number) returns number
end

function heaviestMstEdge(adj list number, n number) returns number
end

function isPrimConnected(adj list number, n number) returns bool
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/graphprim` (tier complex, category complex). Exports 7 function(s):
primMst, primParents, primOr

```
`import "std/c/graphprim" . `use graphprim`  
- `primMst(adj list number, n number) ? number`  
- `primParents(adj list number, n number) ? list number`  
- `primOrder(adj list number, n number) ? list number`  
- `mstEdgeCount(adj list number, n number) ? number`  
- `lightestEdge(adj list number, n number) ? number`  
- `heaviestMstEdge(adj list number, n number) ? number`  
- `isPrimConnected(adj list number, n number) ? bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphprim"  
use graphprim
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? primMst(adj list number, n number) ? number  
? primParents(adj list number, n number) ? list  
? primOrder(adj list number, n number) ? list  
? mstEdgeCount(adj list number, n number) ? number  
? lightestEdge(adj list number, n number) ? number  
? heaviestMstEdge(adj list number, n number) ? number  
? isPrimConnected(adj list number, n number) ? bool
```

4. Usage examples

```
import "std/c/graphprim"  
use graphprim  
  
create result = primMst(/* args */)   
say result  
  
create result = primParents(/* args */)   
say result  
  
create result = primOrder(/* args */)   
say result  
  
create result = mstEdgeCount(/* args */)   
say result  
  
create result = lightestEdge(/* args */)   
say result  
  
create result = heaviestMstEdge(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/graphprim"` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module graphprim
```

```
function primMst(adj list number, n number) returns number
end

function primParents(adj list number, n number) returns list number
end

function primOrder(adj list number, n number) returns list number
end

function mstEdgeCount(adj list number, n number) returns number
end

function lightestEdge(adj list number, n number) returns number
end

function heaviestMstEdge(adj list number, n number) returns number
end

function isPrimConnected(adj list number, n number) returns bool
end
end
```